

# 11

## VISÃO COMPUTACIONAL: MÁQUINAS COM OLHOS E INTUIÇÃO

---

## LISTA DE FIGURAS

|                                                                                                                                                                                                                    |    |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figura 1 - Imagem de um gato no sofá. À esquerda, como um humano vê a imagem. À direita, como um computador lê uma imagem .....                                                                                    | 6  |
| Figura 2 - Imagem de uma maçã .....                                                                                                                                                                                | 7  |
| Figura 3 - Detecção e reconhecimento de objetos em imagens. Acima, imagem do trânsito em uma cidade. Abaixo, veículos, placas e pessoas sendo detectadas e reconhecidas por algoritmo de Visão Computacional ..... | 8  |
| Figura 4 - Segmentação de objetos em imagens. Acima, imagem do trânsito em uma cidade. Abaixo, veículos, placas, calçadas e pessoas sendo segmentadas por algoritmo de Visão Computacional .....                   | 9  |
| Figura 5 - Fósseis dos primeiros animais com olhos.....                                                                                                                                                            | 11 |
| Figura 6 - Evolução dos modelos no Benchmark ImageNet.....                                                                                                                                                         | 14 |
| Figura 7 - Detecção de objetos em imagens .....                                                                                                                                                                    | 15 |
| Figura 8 - Imagem em escala de cinza .....                                                                                                                                                                         | 19 |
| Figura 9 - Imagem RGB .....                                                                                                                                                                                        | 20 |

## LISTA DE CÓDIGOS-FONTE

|                                                                  |    |
|------------------------------------------------------------------|----|
| Código-fonte 1 - Lendo imagens com o OpenCV .....                | 27 |
| Código-fonte 2 - Salvando uma imagem com OpenCV.....             | 28 |
| Código-fonte 3 - Visualizando canais de cores separadamente..... | 29 |
| Código-fonte 4 - Acessando cada pixel individualmente .....      | 30 |
| Código-fonte 5 - Obtendo as dimensões da imagem .....            | 31 |
| Código-fonte 6 - Definindo uma região de interesse .....         | 31 |
| Código-fonte 7 - Conversão de canais de cores .....              | 32 |
| Código-fonte 8 - Aplicação de filtros de suavização .....        | 33 |
| Código-fonte 9 - Detecção de bordas.....                         | 34 |
| Código-fonte 10 - Transformações geométricas .....               | 35 |
| Código-fonte 11 - Adição de anotações nas imagens .....          | 36 |

## SUMÁRIO

|                                                            |    |
|------------------------------------------------------------|----|
| 1 VISÃO COMPUTACIONAL: MÁQUINAS COM OLHOS E INTUIÇÃO ..... | 5  |
| 1.1 O que é Visão Computacional? .....                     | 5  |
| 1.2 Principais Objetivos da Visão Computacional .....      | 8  |
| 1.2.1 Detecção e reconhecimento .....                      | 8  |
| 1.2.2 Segmentação .....                                    | 9  |
| 1.2.3 Interpretação de movimento e contexto .....          | 9  |
| 1.3 Um Breve Histórico .....                               | 10 |
| 1.3.1 A História da Visão .....                            | 10 |
| 1.4 Aplicações de Visão Computacional .....                | 15 |
| 2 PROCESSAMENTO DIGITAL DE IMAGENS .....                   | 19 |
| 2.1 Imagem Digital .....                                   | 19 |
| 2.2 Laboratório OpenCV .....                               | 24 |
| 2.3 Primeiros Passos com OpenCV .....                      | 27 |
| CONCLUSÃO .....                                            | 37 |
| REFERÊNCIAS .....                                          | 38 |

## 1 VISÃO COMPUTACIONAL: MÁQUINAS COM OLHOS E INTUIÇÃO

Você já se perguntou como o celular reconhece seu rosto para desbloquear a tela? Ou como o Google Fotos identifica seu cachorro em milhares de imagens? Essas habilidades vêm de um campo da Inteligência Artificial chamado **Visão Computacional**. Nesta área, ensinamos as máquinas a “ver” e “entender” o mundo visual como os humanos fazem.

Neste capítulo, iremos explorar os fundamentos e a evolução da Visão Computacional. Começaremos com uma viagem no tempo, revisitando os marcos históricos que definiram o desenvolvimento dessa tecnologia, desde os primeiros algoritmos baseados em regras até os sistemas modernos impulsionados por Redes Neurais Profundas. Você entenderá como o avanço do poder computacional, o crescimento de bases de dados de imagens — como o ImageNet — e o surgimento do Deep Learning foram fundamentais nessa transformação.

Também introduziremos os conceitos básicos de Processamento Digital de Imagens, uma etapa essencial para preparar os dados visuais e permitir que os algoritmos os analisem com eficiência. Vamos explorar técnicas como filtragem, segmentação, detecção de bordas e transformação de cores, que são fundamentais para diversas aplicações, desde diagnósticos médicos até veículos autônomos.

Ao longo deste capítulo, você verá exemplos práticos, experimentos visuais e aplicações reais que mostram o impacto da Visão Computacional no nosso dia a dia. Faremos isso de forma acessível, conectando teoria e prática para que você não apenas compreenda os conceitos, mas também saiba aplicá-los na resolução de problemas do mundo real.

### 1.1 O que é Visão Computacional?

A Visão Computacional é uma área da Inteligência Artificial que busca ensinar os computadores a interpretar e compreender o mundo visual. Em outras palavras, é o campo que estuda como fazer com que os computadores “vejam” e tomem decisões com base em imagens ou vídeos, assim como nós, humanos, fazemos com olhos e cérebro.

Mas o que exatamente significa “ver” para um computador? Para nós, humanos, enxergar é algo tão natural que nem pensamos a respeito. Vemos uma pessoa e automaticamente reconhecemos seu rosto, identificamos emoções, percebemos se está se aproximando ou se afastando. Isso tudo parece fácil porque o nosso cérebro é incrivelmente bom em interpretar informações visuais. Para os computadores, no entanto, isso é um grande desafio.

Vamos imaginar uma cena simples: um gato dormindo em um sofá. Para uma pessoa, identificar o gato, o sofá e, talvez, até a marca da almofada é uma tarefa quase instantânea. Nosso cérebro está treinado com anos de experiência visual. Ele entende o contexto, reconhece padrões e compara com memórias passadas.

Para um computador, essa mesma cena é representada como uma matriz de milhões de números, correspondendo aos valores de cor de cada pixel. A Visão Computacional tenta dar sentido a esses números. Como reconhecer que determinado conjunto de formas e cores representa um "gato" e não uma "almofada"? Como saber se o animal está realmente dormindo, e não apenas parado?

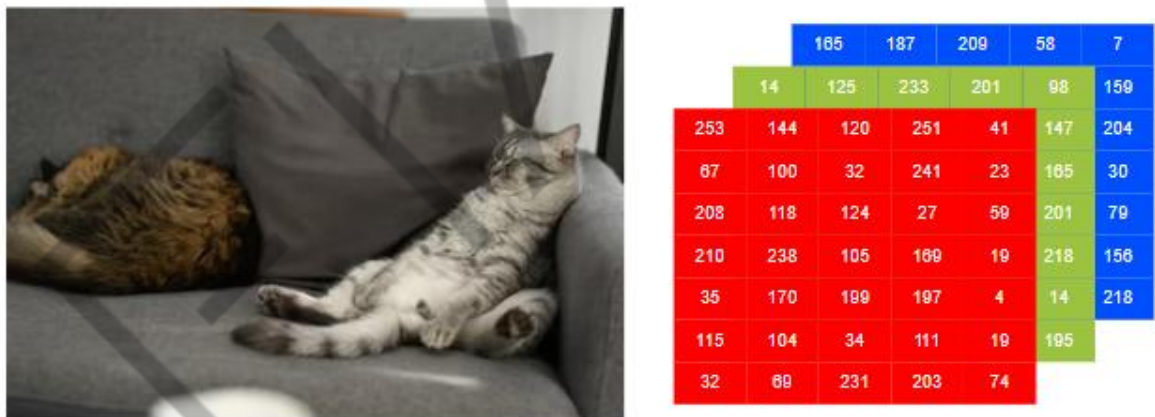


Figura 1 - Imagem de um gato no sofá. À esquerda, como um humano vê a imagem. À direita, como um computador lê uma imagem

Fonte: Freepik e Medium, adaptado pelo autor (2025)

A Visão Computacional resolve esses problemas utilizando uma combinação de Matemática, Estatística, Aprendizado de Máquina e Redes Neurais Profundas (Deep Learning).

Para compreendermos melhor como a Visão Computacional funciona, podemos pensar em um simples exercício. Considere a imagem de uma maçã.

Agora, responda: **Quais características nessa imagem te fazem ter certeza de que isso é uma maçã?**



Figura 2 - Imagem de uma maçã  
Fonte: Google imagens (2025)

Talvez você pense no formato arredondado, na cor vermelha ou verde, no brilho característico da casca, ou até mesmo no cabinho no topo da fruta. De forma quase instantânea, seu cérebro integra todos esses elementos e ativa um “modelo mental” que diz: “isso é uma maçã”.

Esse processo ocorre porque, ao longo da vida, fomos expostos a diversas maçãs em diferentes contextos, em fotos, ao vivo, em desenhos, comidas etc. Com o tempo, nosso cérebro desenvolveu um sistema sofisticado de reconhecimento baseado em padrões visuais. Essa habilidade é resultado de experiências acumuladas e da capacidade do cérebro de generalizar a partir de exemplos.

Os modelos de Aprendizado de Máquina, especialmente as Redes Neurais, funcionam de maneira surpreendentemente semelhante. Eles são alimentados com milhares (ou até milhões) de exemplos de imagens rotuladas. A cada nova imagem, o modelo ajusta seus “pesos” internos para aprender padrões visuais que caracterizam diferentes objetos. Com o tempo, ele constrói uma representação interna desses padrões, assim como fazemos mentalmente, e passa a identificar objetos com boa precisão.

A diferença é que, enquanto nosso cérebro é resultado de milhões de anos de evolução, os modelos de Visão Computacional tentam imitar essa habilidade com



dados e matemática, muitas vezes com excelente desempenho, mas ainda longe da flexibilidade e robustez da visão humana.

### 1.2 Principais Objetivos da Visão Computacional

O principal objetivo da Visão Computacional é compreender o que está acontecendo em uma imagem. Para isso, podemos dividir este objetivo principal em três grupos principais:

#### 1.2.1 Detecção e reconhecimento

Reconhecer e localizar objetos ou padrões em imagens é uma das tarefas mais comuns. Isso inclui aplicações como reconhecimento facial, leitura de placas de carro (Optical Character Recognition — OCR), identificação de produtos em prateleiras, detecção de células cancerígenas em exames de imagem, entre outros.



Figura 3 - Detecção e reconhecimento de objetos em imagens. Acima, imagem do trânsito em uma cidade. Abaixo, veículos, placas e pessoas sendo detectadas e reconhecidas por algoritmo de Visão Computacional

Fonte: Visão Computacional (2022)



### 1.2.2 Segmentação

Segmentar uma imagem significa separar suas partes em regiões com características semelhantes. Por exemplo, separar o fundo da imagem de uma pessoa ou destacar os diferentes órgãos em um exame de ressonância magnética. A sua principal diferença em relação à Detecção de Objetos pode ser observada na figura anterior e na figura seguinte. Enquanto os algoritmos de Detecção de Objetos identificam objetos por meio de uma “caixa retangular”, os algoritmos de Segmentação são mais precisos e conseguem destacar exatamente o contorno de cada objeto na imagem.

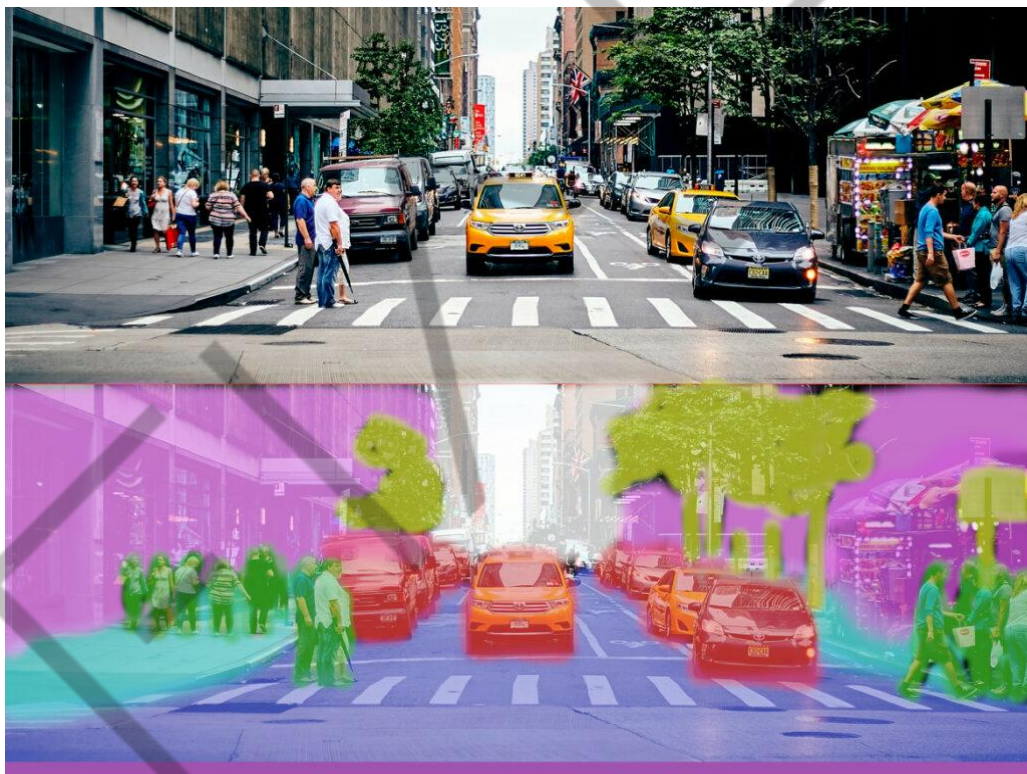


Figura 4 - Segmentação de objetos em imagens. Acima, imagem do trânsito em uma cidade. Abaixo, veículos, placas, calçadas e pessoas sendo segmentadas por algoritmo de Visão Computacional  
Fonte: Visão Computacional (2022)

### 1.2.3 Interpretação de movimento e contexto

Ao aplicar algoritmos como Detecção e Segmentação de objetos em sequências de imagens (como em vídeos), é possível estimar movimentos, rastrear objetos ao longo do tempo e até prever suas trajetórias. Esses algoritmos, quando integrados à pipelines complexos de processamento de imagens, são capazes de

fornecer soluções eficazes para um dos principais desafios da Visão Computacional: entender, em detalhe, o que está acontecendo nas imagens. Isso é essencial para diversas aplicações como carros e robôs autônomos, navegação de drones, entre outras.

### 1.3 Um Breve Histórico

A Visão Computacional evoluiu ao longo das décadas. Antigamente, ela se baseava em regras fixas e algoritmos matemáticos criados artesanalmente. Com o tempo, o Aprendizado de Máquina ganhou força, permitindo que os sistemas aprendessem a reconhecer padrões de forma mais autônoma a partir de dados.

Hoje, a maior revolução da área é o uso de Redes Neurais Convolucionais (CNNs), que imitam o funcionamento do cérebro humano no processamento de imagens. Essas redes conseguem extrair características automaticamente e são a base de aplicações modernas de reconhecimento facial, filtros de redes sociais, diagnóstico por imagem na medicina, carros autônomos, entre muitas outras.

Para compreender melhor a evolução desses algoritmos, faremos uma rápida passagem pelos principais eventos históricos e algoritmos que nortearam o desenvolvimento da área.

#### 1.3.1 A História da Visão

Imagine um mundo primitivo, escuro e silencioso, onde a vida era simples, sem olhos, e se escondia sob o mar. Nesse período, os seres vivos existentes na Terra não possuíam visão. Esse mundo mudou radicalmente há cerca de 543 milhões de anos, durante um evento chamado Explosão Cambriana, um momento único na história da vida em que os organismos começaram a se diversificar rapidamente. Um dos maiores responsáveis por essa explosão de biodiversidade foi o advento da visão.

Antes desse período, a maioria dos organismos marinhos era cega ou apenas sensível à luz. Mas então, pequenos organismos começaram a desenvolver fotorreceptores, células capazes de detectar luz. Com o tempo, essas células foram se organizando em estruturas mais complexas dando origem aos primeiros olhos. A

habilidade de enxergar o ambiente trouxe uma vantagem evolutiva gigantesca: os predadores conseguiam localizar presas com mais precisão, e as presas podiam fugir com mais agilidade. Começou, então, uma corrida evolutiva muito mais dinâmica e acelerada, gerando uma grande quantidade de novas espécies de animais em um curto espaço de tempo.

Essa mudança foi tão significativa que muitos cientistas a chamam de “**Big Bang da Visão**”. A capacidade de ver moldou o caminho da evolução: os animais passaram a ter olhos de diferentes formatos e capacidades, olhos compostos, como os dos insetos; olhos com lentes, como os dos humanos; e até olhos que veem em infravermelho, como algumas serpentes.

## Evolution's Big Bang



This image is licensed under CC-BY 2.5



This image is licensed under CC-BY 2.5



This image is licensed under CC-BY 3.0

### 543million years, B.C.

Figura 5 - Fósseis dos primeiros animais com olhos

Fonte: Blog CSDN (2018)

Avançando milhões de anos no tempo, chegamos à década de 1950, quando dois neurocientistas, David Hubel e Torsten Wiesel, começaram a desvendar o maior mistério da visão: como o cérebro entende o que vemos. Já se compreendia, na época, que os olhos captavam a luz e a convertiam em sinais elétricos. Porém, o que acontecia depois? Como o cérebro transformava esses sinais em imagens, formas, cores e movimento? Hubel e Wiesel começaram seus experimentos com gatos. Eles colocaram pequenos eletrodos na área visual do cérebro dos animais e mostraram diferentes estímulos visuais, como linhas, barras, luzes em movimento. Descobriram

que os neurônios dessa região respondem seletivamente a certos padrões, como linhas verticais ou horizontais e movimentos em direções específicas.

Essa descoberta levou à ideia de que o cérebro processa a visão em camadas. Cada camada do córtex visual analisa um aspecto diferente da imagem: primeiro, pontos de luz; depois, bordas e formas; depois, movimento, profundidade e até rostos.

Esses estudos renderam a Hubel e Wiesel o Prêmio Nobel de Medicina em 1981, e abriram caminho para tudo que conhecemos hoje sobre Visão Computacional, Redes Neurais Convolucionais e como as máquinas aprendem a ver.

A ideia de ensinar máquinas a enxergar remonta às décadas de 1950 e 1960. Naquela época, os computadores eram extremamente limitados, e os algoritmos de visão eram baseados principalmente em regras fixas, muito simples, como comparar intensidades de pixels ou detectar contornos.

Nos anos 60, a Visão Computacional se originou em laboratórios de Inteligência Artificial com projetos ambiciosos como o “Summer Vision Project” do MIT, que tentava descrever cenas tridimensionais a partir de imagens 2D. O progresso foi muito mais difícil do que os cientistas esperavam. Ainda assim, foram feitas descobertas importantes sobre como extrair bordas, contornos e formas básicas das imagens.

Na década de 1980, Yann LeCun desenvolveu as primeiras versões das Redes Neurais Convolucionais, um tipo de arquitetura inspirada no córtex visual de animais. Seu trabalho culminou no desenvolvimento do LeNet, algoritmo utilizado para reconhecimento de dígitos manuscritos, como os encontrados em cheques bancários. Embora promissoras, essas redes exigiam muito poder computacional e grandes volumes de dados, recursos escassos na época. Por isso, as CNNs ficaram relativamente adormecidas até a revolução do Deep Learning nos anos 2000.

Durante os anos 90, surgiram algoritmos baseados em filtragem de imagem com regras matemáticas que se tornaram clássicos, como:

- SIFT (Scale-Invariant Feature Transform) e SURF (Speeded-Up Robust Features), utilizados para detectar e descrever pontos de interesse em imagens.
- Algoritmos de detecção de bordas como o Canny Edge Detector.

- Uso mais robusto de modelos estatísticos, como classificadores baseados em histogramas e Redes Neurais simples.

Apesar dessas contribuições importantes, a área carecia de uma forma padronizada de medir o progresso dos algoritmos. Em 2009, Fei-Fei Li e sua equipe da Universidade de Princeton (que, posteriormente, migraram para a Universidade de Stanford) anunciaram o projeto ImageNet, um banco de dados com mais de 14 milhões de imagens, cada uma rotulada com uma ou mais categorias, inspiradas na hierarquia do WordNet. O diferencial do ImageNet não foi apenas sua escala, mas o fato de fornecer dados anotados de alta qualidade em larga escala, viabilizando experimentos que antes eram impensáveis. Em 2010, o ImageNet Large Scale Visual Recognition Challenge (ILSVRC) foi lançado como uma competição anual para classificar imagens em mil categorias diferentes — como carro, avião, cachorro, gato, entre outros —, reunindo os principais pesquisadores de Visão Computacional do mundo na tentativa de criar sistemas inteligentes o suficiente para classificar quase qualquer objeto presente em uma imagem. Até então, como vimos, os algoritmos de Visão Computacional ainda eram dominados por métodos baseados em engenharia manual de atributos (como SIFT, detecção de bordas etc.), ou seja, os algoritmos tinham que ser desenhados e ajustados manualmente para cada aplicação individualmente. Isso fazia com que a Visão Computacional não fosse escalável e viável para grande parte das aplicações.

Em 2012, Alex Krizhevsky, Ilya Sutskever e Geoffrey Hinton apresentaram a AlexNet, uma Rede Neural Convolucional Profunda (baseada na arquitetura de CNNs proposta por Yann Lecun nos anos 80) treinada com o ImageNet. Com oito camadas, ReLU como função de ativação, dropout como regularização e o uso de GPUs (placas de vídeo) para acelerar o treinamento, a AlexNet superou o segundo colocado na competição por uma margem de 10 pontos percentuais no top-5 error.

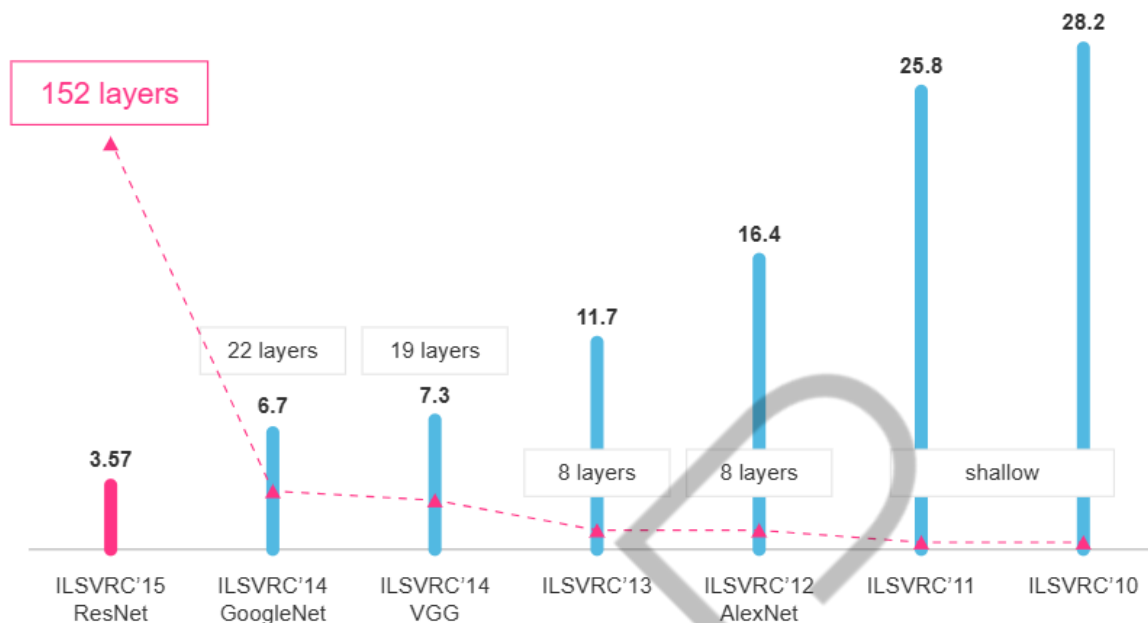


Figura 6 - Evolução dos modelos no Benchmark ImageNet  
Fonte: ResearchGate (2019), adaptado por FIAP (2026)

Esse resultado marcou o início da era moderna da Visão Computacional e desencadeou um verdadeiro “boom” no uso de Deep Learning, com algoritmos de Redes Neurais Profundas aplicados no processamento de imagens. Desde então, novas arquiteturas de Redes Neurais Profundas foram propostas, reduzindo progressivamente o erro no ImageNet e criando modelos de Inteligência Artificial cada vez mais capazes de compreender o mundo visualmente e tomar decisões autônomas. VGG, GoogleNet e ResNet são exemplos de arquiteturas de CNNs que apresentaram melhorias significativas no contexto do ImageNet, e até os dias de hoje são amplamente utilizadas em algumas aplicações de Visão Computacional.

Até o momento, as Redes Neurais apenas sabiam dizer o que havia em uma imagem (“tem um gato ou não na imagem?”, “tem um carro ou não?”). Mas e se quisermos saber onde está o gato na imagem? Ou quantos gatos tem? Ou ainda: diferenciar cada objeto separadamente? Nesse contexto, surgiram as Redes Neurais especialistas em classificar objetos em uma imagem e detectar a região da imagem que este objeto está contido, os chamados modelos de detecção de objetos como R-CNN, YOLO (You Only Look Once) e Mask R-CNN. Em uma foto com duas pessoas e um cachorro, esses modelos não só sabem dizer “pessoas” e “cachorro”, mas também são capazes de mostrar exatamente onde está cada um deles.



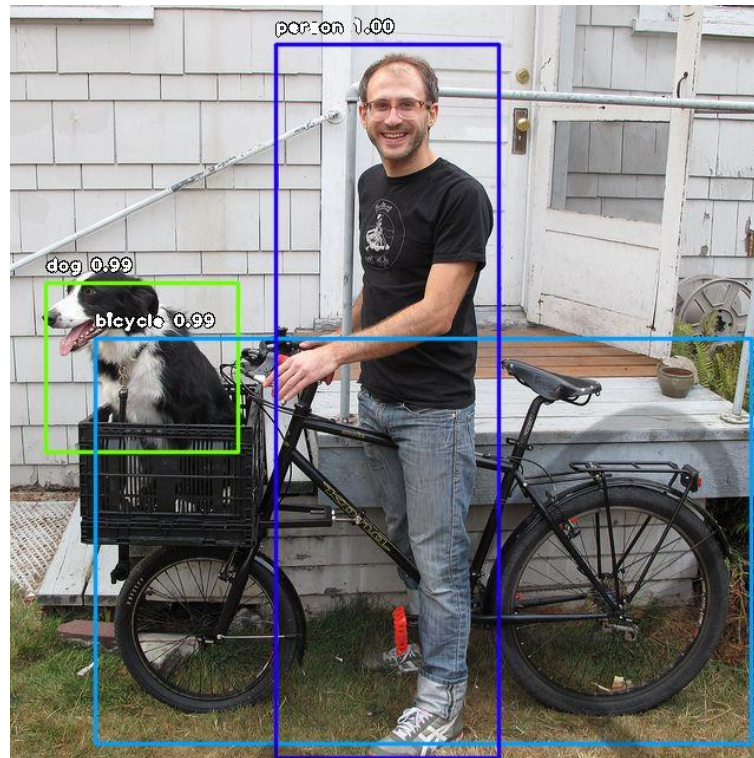


Figura 7 - Detecção de objetos em imagens  
Fonte: Analytics India Magazine (2020)

Em meados de 2020, pesquisadores do Google tiveram a ideia de usar Transformers, uma arquitetura de Redes Neurais muito utilizada em modelos de processamento de texto, como o ChatGPT, para processar imagens. Esse modelo, chamado de Visual Transformers (ViT), permitiu um aumento ainda maior na escalabilidade dos modelos de Visão Computacional, possibilitando o desenvolvimento de sistemas que, além de identificar e detectar objetos, também conseguiam aprender o sentido semântico por trás de uma imagem.

Recentemente, esses modelos foram associados a modelos de processamento de linguagem, resultando em sistemas capazes de descrever todos os detalhes presentes em uma imagem ou até mesmo gerar imagens a partir de textos escritos em linguagem natural. Ao contrário dos modelos anteriores, esses não foram treinados para uma única tarefa — são modelos generalistas, que podem fazer muitas coisas ao mesmo tempo, habilitando uma ampla gama de aplicações.

### 1.4 Aplicações de Visão Computacional

Um dos pilares fundamentais da Visão Computacional reside no reconhecimento e classificação de imagens. Essa capacidade permite que sistemas



identifiquem objetos, pessoas, lugares ou até mesmo emoções presentes em uma imagem. Imagine, por exemplo, um sistema capaz de catalogar automaticamente fotos em uma biblioteca digital, distinguindo entre paisagens, retratos e animais. Essa tecnologia é essencial em ferramentas de busca por imagens na internet, onde algoritmos analisam o conteúdo visual para retornar resultados relevantes às consultas dos usuários. Além disso, sistemas de reconhecimento de imagens são cruciais em aplicações de segurança, como a identificação de objetos suspeitos em aeroportos ou a classificação de imagens médicas para auxiliar no diagnóstico de doenças.

Ao avançar um pouco mais, encontramos a detecção de objetos, que não apenas identifica a presença de um objeto, mas também localiza sua posição exata dentro de uma imagem, geralmente através da criação de caixas delimitadoras ao redor dos objetos detectados. Essa funcionalidade é essencial para: o funcionamento de carros autônomos, que precisam detectar pedestres, outros veículos e placas de trânsito em tempo real para navegar com segurança; sistemas de vigilância, onde a detecção de objetos permite o rastreamento de indivíduos ou veículos em áreas monitoradas, acionando alertas em caso de comportamentos suspeitos; e, na indústria, essa tecnologia é utilizada para identificar peças defeituosas em linhas de produção, garantindo a qualidade dos produtos finais.

Em um nível ainda mais granular, a segmentação de imagens busca particionar a imagem em regiões significativas, atribuindo um rótulo a cada pixel. Existem diferentes tipos de segmentação, como: a segmentação semântica, que agrupa pixels pertencentes à mesma categoria (por exemplo, todos os pixels que representam um carro são rotulados como "carro"); e a segmentação de instâncias, que distingue entre diferentes instâncias do mesmo objeto (por exemplo, cada carro individual em uma imagem é segmentado como uma entidade separada). Essa técnica é fundamental em diversas áreas, tais como: medicina, veículos autônomos e reconhecimento facial. Na área médica, a segmentação de tumores ou órgãos em imagens de ressonância magnética ou tomografia computadorizada auxilia no diagnóstico e no planejamento de tratamentos. Em veículos autônomos, a segmentação de imagens ajuda a entender a cena ao redor, distinguindo a estrada de pedestres, árvores e outros objetos. No reconhecimento facial, essa tecnologia é capaz de identificar ou verificar a identidade

de uma pessoa a partir de uma imagem digital ou vídeo, comparando características faciais extraídas da imagem com um banco de dados de rostos conhecidos.

As aplicações da Visão Computacional são vastas, abrangendo desde o desbloqueio de smartphones e o controle de acesso em edifícios até a identificação de pessoas desaparecidas e a análise de sentimentos em grandes grupos. No entanto, é importante ressaltar as questões éticas e de privacidade associadas ao uso dessa tecnologia, que demandam regulamentações e debates cuidadosos.

Outra área de crescente importância é a estimação de pose, que visa determinar a orientação e a posição de objetos ou figuras humanas em uma imagem ou vídeo. No caso de humanos, essa técnica identifica pontos-chave do corpo (como articulações) e os conecta para formar um esqueleto virtual. Essa tecnologia tem aplicações em diversas áreas, como a análise de movimentos de atletas para melhorar o desempenho, a criação de interfaces de interação homem-computador baseadas em gestos, a animação de personagens virtuais em jogos e filmes, e até mesmo em sistemas de segurança para detectar quedas de idosos.

A capacidade de “ler” texto em imagens, conhecida como Reconhecimento Óptico de Caracteres (OCR), é outra aplicação poderosa da Visão Computacional. Essa tecnologia transforma imagens contendo texto (como documentos digitalizados, placas de trânsito ou legendas em vídeos) em texto editável e pesquisável. O OCR facilita a digitalização e o processamento de grandes volumes de documentos, automatiza a entrada de dados, melhora a acessibilidade para pessoas com deficiência visual (através de leitores de tela) e permite a tradução automática de textos em imagens.

A Visão Computacional também impulsiona a geração e edição de imagens. Algoritmos sofisticados podem criar imagens realistas a partir de descrições textuais, modificar o estilo de uma imagem existente (transformando uma foto em uma pintura, por exemplo), remover ou adicionar objetos em uma cena, e até mesmo restaurar fotos antigas danificadas. Essas tecnologias têm aplicações no campo da arte e do design, na criação de conteúdo para entretenimento e publicidade, e também na reconstrução de cenas de crime para investigações forenses.

A análise de sequências de imagens, ou seja, análise de vídeo, representa um desafio ainda maior para a Visão Computacional, pois envolve o processamento de

informações espaciais e temporais. Essa área é aplicada em diferentes contextos, como sistemas de vigilância, para detectar atividades suspeitas ou contar pessoas; análise de eventos esportivos, para rastrear jogadores e analisar jogadas; monitoramento de tráfego, para controlar o fluxo e identificar acidentes; e também no funcionamento de veículos autônomos, que devem interpretar o fluxo contínuo de informações visuais do ambiente.

No campo da medicina, a Visão Computacional tem se mostrado uma ferramenta promissora. Algoritmos são capazes de analisar imagens médicas (radiografias, tomografias, ressonâncias magnéticas, entre outras) para auxiliar no diagnóstico de diversas doenças, como câncer, retinopatia diabética e doenças neurológicas. Além disso, a Visão Computacional é utilizada no planejamento cirúrgico, na navegação robótica durante procedimentos minimamente invasivos e no monitoramento de pacientes em unidades de terapia intensiva.

A automação industrial e o controle de qualidade também se beneficiam da Visão Computacional. Câmeras e algoritmos analisam produtos em linhas de produção para detectar defeitos, verificar a montagem correta de componentes, inspecionar embalagens e garantir a conformidade com padrões de qualidade. Essa aplicação aumenta a eficiência, reduz custos e melhora a precisão dos processos de fabricação.

As tecnologias de Realidade Aumentada (RA) e Realidade Virtual (RV) dependem fortemente da Visão Computacional para criar experiências imersivas e interativas. Na RA, os algoritmos de Visão Computacional analisam o ambiente real através da câmera de um dispositivo (como um smartphone ou óculos inteligentes) e sobrepõem informações digitais ao mundo real. Na RV, a Visão Computacional pode ser utilizada para rastrear os movimentos do usuário e adaptar o ambiente virtual de acordo.

Em resumo, as aplicações da Visão Computacional são amplas e continuam a se expandir à medida que a tecnologia avança. Desde tarefas básicas como reconhecimento de objetos, até análises complexas de vídeo e aplicações em áreas críticas como a medicina e a indústria, a capacidade das máquinas de “ver” e interpretar o mundo visual está transformando a sociedade de modo promissor. O futuro reserva ainda mais inovações e aplicações surpreendentes para esse campo fascinante da Inteligência Artificial.

## 2 PROCESSAMENTO DIGITAL DE IMAGENS

### 2.1 Imagem Digital

Quando olhamos para uma imagem em nosso dia a dia, seja uma fotografia tirada com o celular, uma pintura digital ou um frame de um vídeo, temos a impressão de que estamos vendo algo contínuo, como se fosse uma representação perfeita da realidade. No entanto, para um computador, uma imagem nada mais é do que uma coleção de números organizada em uma estrutura chamada matriz. Essa representação numérica é o ponto de partida para tudo o que fazemos na Visão Computacional.

Uma imagem digital é composta por pequenos pontos chamados pixels (uma abreviação de “*picture element*”). Cada pixel carrega informações sobre a cor ou a intensidade luminosa em um ponto específico da imagem.

Em uma imagem em tons de cinza, cada pixel é representado por um único número, geralmente entre 0 e 255, onde 0 representa o preto absoluto, 255 representa o branco, e os valores intermediários representam tons de cinza.

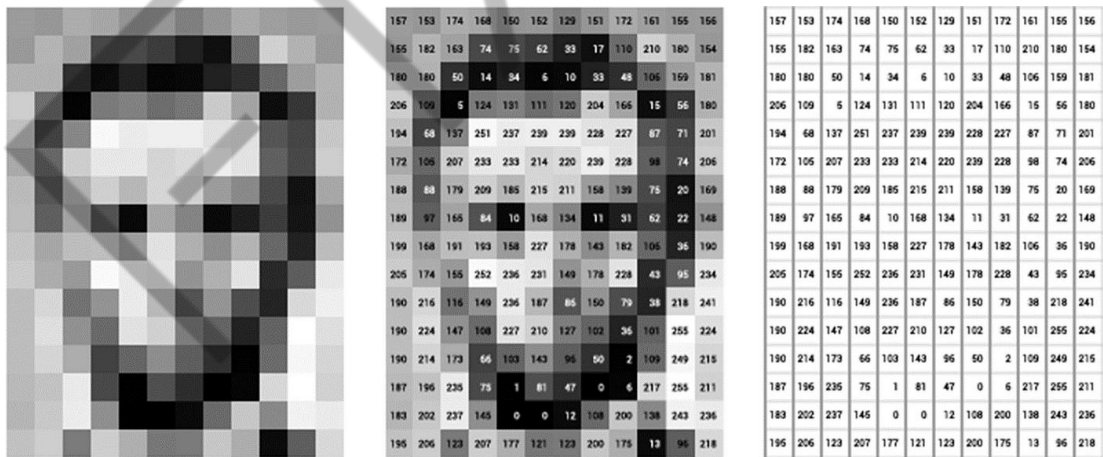


Figura 8 - Imagem em escala de cinza

Fonte: Medium – Decoding Digital Images: The Fundamentals of Greyscale and RGB (2024)

Já em uma imagem colorida, o mais comum é que cada pixel seja representado por uma combinação de três valores que indicam a intensidade das cores primárias da luz: vermelho (R), verde (G) e azul (B). Essa representação é conhecida como o modelo de cor RGB. Por exemplo, um pixel com os valores (255, 0, 0) é totalmente

vermelho, enquanto (0, 255, 0) é totalmente verde e (255, 255, 255) representa o branco.

Para o computador, essa imagem RGB será representada por uma matriz tridimensional: largura  $\times$  altura  $\times$  3. Ou seja, uma imagem com 640 pixels de largura e 480 de altura terá uma matriz com dimensões  $480 \times 640 \times 3$ . Cada célula dessa matriz contém um valor entre 0 e 255, representando a intensidade de uma das três cores básicas naquele ponto.

A resolução de uma imagem, frequentemente expressa em termos de largura e altura (como  $1920 \times 1080$ ), indica o número total de pixels que compõem a imagem. Quanto maior a resolução, maior o número de pixels e, consequentemente, maior o nível de detalhes que a imagem pode exibir.

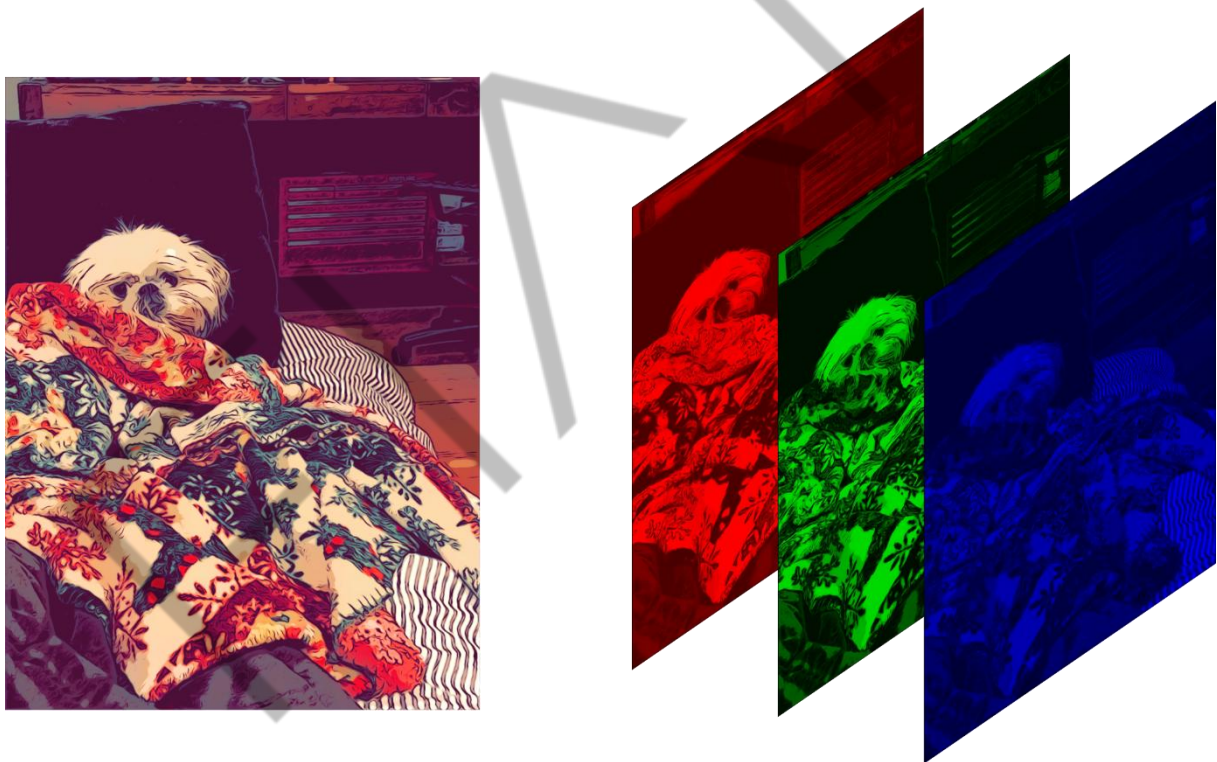


Figura 9 - Imagem RGB

Fonte: Medium – Decoding Digital Images: The Fundamentals of Greyscale and RGB (2024)

Mas como um computador, uma máquina essencialmente lógica e numérica, interpreta essa coleção de números para “entender” uma imagem?

A compreensão da imagem por um computador envolve o processamento desses dados numéricos através de algoritmos específicos. Inicialmente, o computador pode realizar operações básicas, como ler os valores dos pixels, identificar padrões de intensidade ou cores adjacentes, e realizar transformações

matemáticas nesses valores. À medida que os algoritmos se tornam mais complexos, o computador pode começar a identificar características mais significativas na imagem, como bordas, cantos, texturas e formas. Através de técnicas avançadas de Aprendizado de Máquina, o computador pode ser treinado para reconhecer objetos específicos, rostos, cenas e até mesmo interpretar o significado semântico do conteúdo visual.

Em essência, a Visão Computacional busca construir um modelo computacional do sistema visual humano, permitindo que as máquinas extraiam informações úteis e tomem decisões com base em dados visuais.

O processamento de imagens digitais constitui a base para muitas tarefas da Visão Computacional, envolvendo uma série de operações que modificam a imagem de entrada para produzir uma imagem de saída com características alteradas ou realçadas. Essas operações podem variar desde ajustes simples de brilho e contraste até transformações complexas que alteram a geometria ou o conteúdo da imagem. Compreender a teoria por trás dessas operações é fundamental para aplicar as ferramentas de forma eficaz.

Uma classe importante de operações é a das **transformações geométricas**. Essas transformações alteram a posição espacial dos pixels na imagem, modificando sua geometria. Imagine uma imagem como uma folha de borracha que pode ser esticada, comprimida ou movida. A **translação**, por exemplo, é a mais simples das transformações geométricas, consistindo em deslocar todos os pixels da imagem por uma quantidade fixa na direção horizontal e/ou vertical. Cada pixel  $(x,y)$  na imagem original é movido para uma nova posição  $(x',y')$  na imagem de saída, onde  $x'=x+\Delta x$  e  $y'=y+\Delta y$ , sendo  $\Delta x$  e  $\Delta y$  os valores de deslocamento. A **rotação**, por sua vez, gira a imagem em torno de um ponto central (geralmente o centro da imagem ou a origem) por um determinado ângulo. Cada pixel é movido para uma nova posição que mantém sua distância ao centro de rotação, mas altera sua orientação angular. O **redimensionamento**, ou escalonamento, altera o tamanho da imagem, expandindo-a ou contraindo-a. Isso é feito multiplicando as coordenadas dos pixels por fatores de escala nas direções horizontal e vertical. Uma questão crucial que surge em transformações geométricas, exceto na translação por valores inteiros, é que os pixels da imagem original podem não cair exatamente nas posições inteiras da grade de pixels na imagem de destino. Nesses casos, é necessário utilizar técnicas de



**interpolação** para estimar o valor (cor ou intensidade) dos pixels na imagem de saída com base nos valores dos pixels vizinhos mais próximos na imagem original. Métodos comuns incluem a interpolação do vizinho mais próximo, bilinear ou bicúbica, cada uma oferecendo um trade-off diferente entre complexidade computacional e qualidade da imagem resultante.

Outra categoria fundamental de operações é a **filtragem de imagens**. A filtragem envolve modificar o valor de cada pixel com base nos valores dos pixels em sua vizinhança. A ideia central é utilizar uma pequena matriz — conhecida como **kernel**, **máscara** ou **janela** — que “desliza” sobre a imagem. Em cada posição, os valores do kernel são multiplicados pelos valores dos pixels da imagem sob ele, e a soma desses produtos determina o novo valor do pixel central na imagem de saída. Este processo matemático é formalmente conhecido como **convolução**. Diferentes kernels são projetados para realizar diferentes tipos de filtragem. Os filtros de **suavização**, como o filtro de média ou o filtro Gaussiano, utilizam kernels com valores positivos que somam 1. Eles tendem a “borrar” a imagem, reduzindo variações bruscas de intensidade, o que é eficaz para diminuir o **ruído** e suavizar as transições. O filtro Gaussiano, em particular, utiliza pesos que diminuem com a distância do centro, imitando uma distribuição Gaussiana, e que resulta em uma suavização mais natural. Em contraste, os filtros de **realce** ou de **detecção de bordas** utilizam kernels com valores positivos e negativos. Esses kernels são projetados para responder a mudanças abruptas de intensidade. Ao passar por uma região de intensidade constante, o resultado da convolução será próximo de zero. No entanto, ao passar por uma borda (onde há uma transição rápida de intensidade), o resultado terá um valor significativo, indicando a presença e a orientação da borda.

A **detecção de bordas** é uma aplicação específica e crucial da filtragem e da análise de gradiente. Bordas representam contornos de objetos ou limites entre diferentes regiões na imagem, sendo informações visuais de alto nível importantes para muitas tarefas de Visão Computacional. Algoritmos de detecção de bordas buscam identificar pixels onde a intensidade muda significativamente. Métodos simples calculam o **gradiente** da imagem, que mede a taxa de variação da intensidade em cada ponto. O gradiente é um vetor que aponta na direção de maior aumento de intensidade, e sua magnitude indica o quão forte é essa mudança. Bordas fortes correspondem a magnitudes de gradiente elevadas. Algoritmos mais sofisticados,

como o detector de bordas Canny, utilizam múltiplos estágios para refinar a detecção. Isso inclui a aplicação inicial de um filtro de suavização para reduzir o ruído, o cálculo do gradiente, a **supressão de não máximos** (que “afina” as bordas, garantindo que apenas os picos locais de gradiente sejam mantidos) e a **histerese por limiar** (que usa dois limiares para conectar segmentos de borda e remover bordas fracas isoladas que provavelmente são ruído).

Outra operação de segmentação fundamental é a **limiarização (thresholding)**. Essa técnica simples, mas poderosa, é frequentemente aplicada a imagens em escala de cinza. Ela compara o valor de intensidade de cada pixel com um valor limite (o threshold). Se o valor do pixel for maior que o limite, o pixel na imagem de saída recebe um valor alto (geralmente 255, representando branco); caso contrário, recebe um valor baixo (geralmente 0, representando preto). O resultado é uma **imagem binária**, contendo apenas dois valores possíveis. A limiarização é extremamente útil para separar objetos de interesse do fundo quando há um contraste razoável entre eles. A escolha do valor limite é crucial e pode ser feita manualmente ou automaticamente através de algoritmos como o método de Otsu, que busca o limiar que melhor separa os pixels em dois grupos com base na análise do histograma da imagem.

As **operações morfológicas** constituem um conjunto de operações não lineares que aplicam um “elemento estruturante” (semelhante a um kernel, mas usado de forma diferente) para modificar a forma dos objetos em imagens, tipicamente binárias. As duas operações morfológicas básicas são a **erosão** e a **dilatação**. A **erosão** “encolhe” os objetos brancos em uma imagem binária, removendo pixels das bordas. Para cada pixel, a erosão verifica se o elemento estruturante cabe completamente dentro da região branca quando centrado naquele pixel; caso contrário, o pixel de saída é preto. Isso pode ser utilizado para remover pequenos ruídos brancos ou separar objetos conectados. A **dilatação**, por outro lado, “expande” os objetos brancos, adicionando pixels às bordas. Para cada pixel, a dilatação verifica se o elemento estruturante toca qualquer parte branca quando centrado naquele pixel; se sim, o pixel de saída é branco. A dilatação pode ser utilizada para preencher pequenos buracos em objetos ou conectar objetos próximos. Operações mais complexas, como **abertura** (erosão seguida de dilatação) e **fechamento** (dilatação

seguida de erosão), combinam as operações básicas para remover ruído e preencher buracos, respectivamente, mantendo a forma geral dos objetos.

Além dessas, existem outras operações importantes, como a **conversão de espaço de cores**, que transforma a representação numérica da cor de uma imagem de um modelo para outro (por exemplo, de BGR para HSV). Isso é útil porque diferentes espaços de cor separam a informação de cor e brilho de maneiras distintas, o que pode ser mais vantajoso para certas análises. Em relação à **análise de histograma**, embora não seja uma operação que modifique a imagem diretamente, é uma ferramenta fundamental para entender a distribuição de intensidade ou cor, sendo frequentemente usada para guiar outras operações, como o ajuste de contraste ou a limiarização.

## 2.2 Laboratório OpenCV

Para explorarmos o vasto mundo da Visão Computacional na prática, vamos introduzir o **OpenCV (Open Source Computer Vision Library)**, uma biblioteca de software de código aberto amplamente utilizada para tarefas de Visão Computacional, Aprendizado de Máquina e processamento de imagens. O OpenCV oferece um conjunto abrangente de funções e algoritmos otimizados, tornando-se uma ferramenta indispensável para pesquisadores, desenvolvedores e entusiastas da área. Sua interface em Python, através da biblioteca cv2, proporciona uma maneira poderosa e concisa de implementar soluções de Visão Computacional.

Para começarmos a utilizar o OpenCV em Python, o primeiro passo é garantir que a biblioteca esteja instalada em seu ambiente de desenvolvimento. Utilizaremos a Plataforma Google Colaboratory (Colab) como principal ambiente de desenvolvimento integrado (IDE).

O Colab é um ambiente Jupyter Notebook gratuito que roda completamente na nuvem, ou seja, você não precisa instalar nada em seu computador para começar a programar em Python e utilizar bibliotecas como o OpenCV. Tudo o que você precisa é de um navegador web e uma conta Google.

Para iniciar sua jornada com o Colab, siga os seguintes passos:

- **Passo 1 – Acesse o Google Colab:** abra seu navegador web preferido (como Chrome, Firefox, Safari, entre outros) e navegue até o endereço <<https://colab.research.google.com/>>. Será necessário logar a sua conta Google.
- **Passo 2 – Crie um novo notebook:** uma vez na página do Google Colab, você verá uma janela de boas-vindas. Para criar um novo ambiente de programação, clique em "Novo notebook" no canto inferior direito da janela ou vá em "Arquivo" no menu superior e selecione "Novo notebook". Uma nova aba será aberta com um notebook em branco, pronto para receber seu código Python.
- **Passo 3 – Ambiente Python:** o Colab já vem com um ambiente Python pré-configurado, isso significa que você pode começar a escrever e executar código Python imediatamente.
- **Passo 4 – Instalando o OpenCV:** como o OpenCV não está instalado por padrão em todos os notebooks Colab, precisaremos instalá-lo. Para fazer isso, digite o seguinte comando em uma célula de código (uma das caixas onde você escreve o código) e pressione Shift + Enter ou clique no botão de "play" ao lado da célula para executar o comando: **!pip install opencv-python**.
  - O ponto de exclamação ! no início do comando indica que ele deve ser executado no shell do sistema operacional, e não como código Python. Este comando instruirá o pip, o gerenciador de pacotes do Python, a baixar e instalar a biblioteca opencv-python. Você verá uma saída de texto indicando o progresso da instalação.
- **Passo 5 – Executando células de código:** no Colab, você deverá escrever e executar seu código em células. Existem as células de código (onde você escreve o código Python) e células de texto (onde você pode escrever explicações, como esta). Para executar uma célula de código, selecione-a e pressione Shift + Enter ou clique no ícone de "play" que aparece quando você passa o mouse sobre a célula. A saída do código (se houver) será exibida abaixo da célula.

- **Paso 6 – Carregando imagens:** para trabalhar com suas próprias imagens, você precisará carregá-las no ambiente do Colab. Existem algumas maneiras de fazer isso:
  - **Upload direto:** no painel esquerdo do Colab, clique no ícone de pasta ("Arquivos"). Você verá uma opção para "Fazer upload". Clique nela e selecione as imagens que você deseja usar do seu computador. As imagens carregadas ficarão visíveis na aba "Arquivos". Você poderá referenciá-las no seu código utilizando o nome do arquivo (por exemplo, 'minha\_imagem.jpg').
  - **Conectando ao Google Drive:** caso suas imagens estiverem armazenadas no Google Drive, você pode conectar seu notebook Colab ao seu Drive. No menu superior, vá em "Arquivo" > "Montar Drive". Siga as instruções para autorizar o Colab a acessar seus arquivos do Drive. Uma pasta chamada "drive" será criada no painel esquerdo, e você poderá navegar até suas imagens dentro dela. O caminho para encontrar uma imagem no seu Drive seria algo como: `/content/drive/MyDrive/pasta_das_imagens/minha_imagem.jpg`.
- **Passo 7 – Salvando seu trabalho:** o Colab salva automaticamente seu notebook a cada poucos segundos. Você também pode salvar manualmente indo em "Arquivo" > "Salvar". Para baixar o notebook para o seu computador, vá em "Arquivo" > "Fazer download" > "Fazer download de .ipynb". É possível baixar arquivos que criou (como imagens processadas) acessando a aba "Arquivos" no painel esquerdo, encontrando o arquivo, clicando nos três pontos ao lado dele e selecionando "Fazer download".

Com estes passos iniciais, você estará pronto para configurar o Google Colab e começar a experimentar com os exemplos de código e os conceitos de Visão Computacional que estamos explorando. O Colab oferece um ambiente flexível e poderoso para aprender e praticar Visão Computacional sem a necessidade de configurações complexas em seu próprio computador.

## 2.3 Primeiros Passos com OpenCV

Após a instalação bem-sucedida, podemos começar a interagir com imagens utilizando o OpenCV. Um dos primeiros passos é carregar uma imagem do disco. A função `cv2.imread()` é utilizada para essa finalidade. Ela recebe como argumento o caminho para o arquivo da imagem e retorna uma representação da imagem como um array NumPy.

```
import cv2

# Carrega uma imagem chamada 'minha_imagem.jpg'
imagem = cv2.imread('minha_imagem.jpg')
print(imagem)
# Verifica se a imagem foi carregada corretamente
if imagem is None:
    print("Erro ao carregar a imagem.")
else:
    print(f"Dimensões da imagem: {imagem.shape}")

# Mostra a imagem
cv2.imshow('Imagem Original', imagem)
```

Código-fonte 1 - Lendo imagens com o OpenCV  
Fonte: Elaborado pelo autor (2025)

Neste exemplo, `cv2.imread('minha_imagem.jpg')` tenta carregar a imagem especificada. A variável `imagem` conterá um array NumPy representando a imagem se a operação for bem-sucedida. Caso contrário, ela será `None`. É sempre uma boa prática verificar se a imagem foi carregada corretamente antes de prosseguir com qualquer processamento. O atributo `.shape` do array NumPy retorna uma tupla contendo as dimensões da imagem (altura, largura e número de canais de cor). Para uma imagem colorida no modelo RGB, o número de canais será 3. Uma vez que a imagem está carregada na memória, podemos visualizá-la utilizando a função `cv2.imshow()`.

Também é possível salvar uma imagem processada ou modificada utilizando a função `cv2.imwrite()`. Esta função recebe como argumentos o caminho para o arquivo onde a imagem será salva e o array NumPy representando a imagem.

```
import cv2

# Carrega a imagem
imagem = cv2.imread('minha_imagem.jpg')

if imagem is not None:
    # Cria uma cópia da imagem (apenas para este exemplo)
    imagem_copia = imagem.copy()

    # Salva a cópia da imagem com o nome 'imagem_copia.png'
    cv2.imwrite('imagem_copia.png', imagem_copia)

    print("Imagem salva com sucesso!")
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 2 - Salvando uma imagem com OpenCV  
Fonte: Elaborado pelo autor (2025)

Neste exemplo, carregamos a imagem, criamos uma cópia e a salvamos em um novo arquivo chamado `imagem_copia.png`. O OpenCV suporta diversos formatos de imagem, como JPEG, PNG, TIFF etc., dependendo das bibliotecas instaladas em seu sistema.

Como vimos anteriormente, uma imagem é representada como um array NumPy. Podemos acessar um pixel específico utilizando suas coordenadas de linha (altura) e coluna (largura). Para uma imagem colorida, podemos também decompor cada canal de cor (0 para azul, 1 para verde e 2 para vermelho, na ordem BGR do OpenCV).



```
import matplotlib.pyplot as plt

# Definindo a imagem
fig = plt.figure(figsize=(12, 12));

# Separando apenas o componente Vermelho (Blue)
ax = fig.add_subplot(1, 3, 1);
ax.set_title('Blue');
ax.imshow(imagem[:, :, 0], cmap="gray");
ax.set_xticks([]); ax.set_yticks([]);

# Separando apenas o componente Verde (Green)
ax = fig.add_subplot(1, 3, 2);
ax.set_title('Green');
ax.imshow(imagem[:, :, 1], cmap="gray");
ax.set_xticks([]); ax.set_yticks([]);

# Separando apenas o componente Azul (Red)
ax = fig.add_subplot(1, 3, 3);
ax.set_title('Red');
ax.imshow(imagem[:, :, 2], cmap="gray");
ax.set_xticks([]); ax.set_yticks([]);
```

Código-fonte 3 - Visualizando canais de cores separadamente  
Fonte: Elaborado pelo autor (2025)

Este bloco de código tem como objetivo principal decompor uma imagem colorida nos seus canais de cor individuais — vermelho, verde e azul — e exibi-los lado a lado para análise visual. Para que este código funcione, é necessário ter importado a biblioteca Matplotlib (usualmente como: **import matplotlib.pyplot as plt**).

A primeira linha, `fig = plt.figure(figsize=(12, 12));`, inicia o processo criando uma “figura” no Matplotlib. O argumento `figsize=(12, 12)` define o tamanho dessa figura em polegadas, especificando que ela terá 12 polegadas de largura por 12 polegadas de altura. A linha mais crucial para a visualização da imagem é `ax.imshow(frame[:, :, 0], cmap="gray");`. O argumento `frame[:, :, 0]` é o responsável por selecionar o canal de cor que será exibido. Assumindo que `frame` é o array NumPy que representa a imagem carregada pelo OpenCV, o `frame[:, :, 0]` utiliza a notação de slicing do NumPy. Os dois pontos (:) em `[:, :, 0]` significam “selecione todas as linhas” e “selecione todas as colunas”. O 0 (zero) no final indica que estamos selecionando o primeiro canal de cores da imagem. É importante lembrar que, por padrão, o OpenCV carrega imagens na ordem de canais BGR (azul, verde, vermelho), e não RGB. Portanto, `frame[:, :, 0]` está na verdade acessando o canal azul.

Também podemos acessar cada pixel individualmente para fazer modificações em seus valores.

```
import cv2

# Carrega a imagem
imagem = cv2.imread('minha_imagem.jpg')

if imagem is not None:
    # Acessa o pixel na linha 100, coluna 150 (lembrando que
    # a indexação começa em 0)
    pixel = imagem[100, 150]
    print(f"Valor do pixel em (100, 150): {pixel}")

    # Acessa o componente azul do pixel
    azul = imagem[100, 150, 0]
    print(f"Componente azul do pixel: {azul}")

    # Modifica o pixel para a cor branca (BGR: [255, 255,
    255])
    imagem[100, 150] = [255, 255, 255]

    # Exibe a imagem com o pixel modificado
    cv2.imshow('Imagem Modificada', imagem)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 4 - Acessando cada pixel individualmente  
Fonte: Elaborado pelo autor (2025)

Neste trecho de código, é demonstrado como acessar os valores dos componentes de cor de um pixel específico e como modificar esse pixel para uma nova cor. Embora a manipulação de pixels individuais seja possível, para operações em larga escala, é geralmente mais eficiente utilizar as funcionalidades de processamento de arrays NumPy.

Além de acessar pixels individuais, muitas vezes precisamos obter informações gerais sobre a imagem, como sua largura, altura, número de canais e tipo de dados armazenados. Podemos acessar essas informações através do atributo `.shape` (como já vimos) e do atributo `.dtype`.

```
import cv2

# Carrega a imagem
imagem = cv2.imread('minha_imagem.jpg')

if imagem is not None:
    altura, largura, canais = imagem.shape
    print(f"Altura: {altura}, Largura: {largura}, Canais: {canais}")
    print(f"Tipo de dados da imagem: {imagem.dtype}")
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 5 - Obtendo as dimensões da imagem  
Fonte: Elaborado pelo autor (2025)

Outro conceito importante é o de Região de Interesse (ROI). Uma ROI é uma área específica dentro de uma imagem na qual desejamos focar nossas operações. Podemos definir uma ROI utilizando o mecanismo de slicing do NumPy nos arrays de imagem.

```
import cv2

# Carrega a imagem
imagem = cv2.imread('minha_imagem.jpg')

if imagem is not None:
    # Define uma ROI: linhas de 50 a 150, colunas de 100 a 200
    roi = imagem[50:151, 100:201]

    # Exibe a ROI
    cv2.imshow('ROI', roi)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    # Podemos também modificar a ROI
    roi[:] = [0, 255, 0] # Define a ROI para a cor verde

    # Exibe a imagem com a ROI modificada
    cv2.imshow('Imagem com ROI Modificada', imagem)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 6 - Definindo uma região de interesse  
Fonte: Elaborado pelo autor (2025)

Neste exemplo, extraímos uma região específica da imagem e a exibimos. Em seguida, demonstramos como modificar todos os pixels dentro dessa ROI para uma cor específica.

Frequentemente, em tarefas de Visão Computacional, é útil converter uma imagem colorida para escala de cinza. Uma imagem em escala de cinza possui apenas um canal de intensidade, o que simplifica muitos algoritmos de processamento. O OpenCV fornece a função `cv2.cvtColor()` para realizar conversões entre diferentes espaços de cores.

```
import cv2

# Carrega a imagem colorida
imagem_colorida = cv2.imread('minha_imagem.jpg')

if imagem_colorida is not None:
    # Converte a imagem para escala de cinza
    imagem_cinza = cv2.cvtColor(imagem_colorida,
                                cv2.COLOR_BGR2GRAY)

    # Exibe a imagem em escala de cinza
    cv2.imshow('Imagem em Escala de Cinza', imagem_cinza)
    cv2.waitKey(0)
    cv2.destroyAllWindows()

    # Salva a imagem em escala de cinza
    cv2.imwrite('imagem_cinza.png', imagem_cinza)
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 7 - Conversão de canais de cores  
Fonte: Elaborado pelo autor (2025)

Aqui, utilizamos `cv2.cvtColor()` com o código `cv2.COLOR_BGR2GRAY` para converter a imagem colorida (no espaço de cores BGR) para escala de cinza.

Outro conjunto importante de operações envolve a filtragem de imagens. A filtragem é utilizada para diversas finalidades, como a redução de ruído e o realce de características. Um tipo comum de filtro é a suavização (blur), que ajuda a reduzir o ruído e a suavizar as bordas. O OpenCV oferece diferentes tipos de filtros de suavização, como o filtro de média (`cv2.blur()`) e o filtro gaussiano (`cv2.GaussianBlur()`). Veremos mais detalhes sobre esses filtros nos próximos capítulos.

```
import cv2

# Carrega a imagem
imagem = cv2.imread('minha_imagem.jpg')

if imagem is not None:
    # Aplica um filtro de média com uma janela de 5x5
    imagem_blur = cv2.blur(imagem, (5, 5))

    # Aplica um filtro Gaussiano com um desvio padrão de 0
    imagem_gaussiana = cv2.GaussianBlur(imagem, (5, 5), 0)

    # Exibe as imagens resultantes
    cv2.imshow('Imagem com Blur de Média', imagem_blur)
    cv2.imshow('Imagem com Blur Gaussiano',
    imagem_gaussiana)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 8 - Aplicação de filtros de suavização  
Fonte: Elaborado pelo autor (2025)

O filtro de média calcula a média dos valores dos pixels dentro de uma janela de tamanho especificado e substitui o pixel central por essa média. O filtro Gaussiano utiliza uma função Gaussiana para atribuir pesos diferentes aos pixels vizinhos, resultando em um efeito de suavização mais suave e natural.

Em contraste com a suavização, a detecção de bordas é uma técnica fundamental para identificar as mudanças abruptas na intensidade dos pixels, que geralmente correspondem às bordas dos objetos na imagem. Um dos algoritmos de detecção de bordas mais populares é o detector de bordas Canny (`cv2.Canny()`).

```
import cv2

# Carrega a imagem em escala de cinza
imagem = cv2.imread('minha_imagem.jpg',
cv2.IMREAD_GRAYSCALE)

if imagem is not None:
    # Aplica o detector de bordas Canny
    bordas = cv2.Canny(imagem, 100, 200) # Os valores 100 e
200 são os limiares (thresholds)

    # Exibe a imagem com as bordas detectadas
    cv2.imshow('Bordas Canny', bordas)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 9 - Detecção de bordas  
Fonte: Elaborado pelo autor (2025)

O detector de bordas Canny utiliza um processo multiestágio para detectar uma ampla gama de bordas em imagens. Os parâmetros de limiar (thresholds) controlam a sensibilidade do detector.

As transformações geométricas nos permitem alterar a posição, orientação ou tamanho de uma imagem. Algumas transformações comuns incluem a translação (deslocamento), a rotação e o redimensionamento. O OpenCV fornece funções como `cv2.warpAffine()` para transformações afins (que incluem translação, rotação, escala e cisalhamento) e `cv2.resize()` para redimensionamento.

```
import cv2
import numpy as np

# Carrega a imagem
imagem = cv2.imread('minha_imagem.jpg')

if imagem is not None:
    altura, largura = imagem.shape[:2]

    # Translação: desloca a imagem 100 pixels para a direita
    e 50 pixels para baixo
    matriz_translacao = np.float32([[1, 0, 100], [0, 1,
50]])
    imagem_translacionada = cv2.warpAffine(imagem,
matriz_translacao, (largura + 100, altura + 50))
```

```
# Rotação: rotaciona a imagem em 45 graus no sentido
anti-horário
ponto_central = (largura // 2, altura // 2)
matriz_rotacao = cv2.getRotationMatrix2D(ponto_central,
45, 1) # O último argumento é a escala
imagem_rotacionada = cv2.warpAffine(imagem,
matriz_rotacao, (largura, altura))

# Redimensionamento: redimensiona a imagem para metade
do tamanho original
imagem_redimensionada = cv2.resize(imagem, (largura //
2, altura // 2))

# Exibe as imagens transformadas
cv2.imshow('Imagem Translacionada',
imagem_translacionada)
cv2.imshow('Imagem Rotacionada', imagem_rotacionada)
cv2.imshow('Imagem Redimensionada',
imagem_redimensionada)
cv2.waitKey(0)
cv2.destroyAllWindows()
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 10 - Transformações geométricas  
Fonte: Elaborado pelo autor (2025)

Neste exemplo, demonstramos como realizar translação, rotação e redimensionamento utilizando as funções do OpenCV. Para a translação, criamos uma matriz de transformação que especifica o deslocamento nas direções x e y. Para a rotação, utilizamos a função `cv2.getRotationMatrix2D()` para calcular a matriz de rotação em torno de um ponto central com um determinado ângulo e escala. O redimensionamento é feito diretamente com a função `cv2.resize()`, especificando as novas dimensões desejadas.

Finalmente, muitas vezes precisamos adicionar anotações visuais às imagens, como desenhar formas geométricas ou escrever texto. O OpenCV fornece funções para realizar essas tarefas.



```
import cv2

# Carrega a imagem
imagem = cv2.imread('minha_imagem.jpg')

if imagem is not None:
    # Desenha um retângulo verde com espessura de 2 pixels
    cv2.rectangle(imagem, (50, 50), (200, 200), (0, 255, 0),
    2)

    # Desenha um círculo azul preenchido
    cv2.circle(imagem, (300, 150), 50, (255, 0, 0), -1) # -1
    indica preenchimento

    # Desenha uma linha vermelha com espessura de 3 pixels
    cv2.line(imagem, (10, 300), (400, 350), (0, 0, 255), 3)

    # Adiciona texto à imagem
    fonte = cv2.FONT_HERSHEY_SIMPLEX
    cv2.putText(imagem, 'OpenCV!', (10, 400), fonte, 1,
    (255, 255, 255), 2, cv2.LINE_AA)

    # Exibe a imagem com as anotações
    cv2.imshow('Imagem com Anotações', imagem)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Erro ao carregar a imagem.")
```

Código-fonte 11 - Adição de anotações nas imagens  
Fonte: Elaborado pelo autor (2025)

Agora já sabemos configurar o Google Colab e os comandos básicos para manipular imagens no Python, como ler e escrever imagens, alterar os valores dos pixels, decompor os canais de cores e até aplicar alguns filtros simples como o filtro GaussianBlur.

Nos próximos capítulos, aprenderemos a fazer transformações e filtrações nessas imagens. Técnicas são consideradas a base para a maioria dos algoritmos de Visão Computacional existentes atualmente.

## CONCLUSÃO

Ao longo deste capítulo, fizemos uma jornada pela fascinante trajetória da Visão Computacional. Começamos compreendendo o que significa dar “visão” a uma máquina. Não se trata apenas de fazer com que ela capture imagens. A Visão Computacional vai muito além: ela busca fazer com que sistemas computacionais **entendam o que estão vendo**, sejam capazes de **interpretar padrões, reconhecer objetos, compreender movimentos**, e até **tomar decisões com base em imagens**, da mesma forma (ou melhor) que um ser humano faria.

Nos próximos capítulos, aprofundaremos gradativamente na teoria e prática dos principais algoritmos de Visão Computacional até chegarmos ao estado da arte, mantendo sempre como objetivo principal o de responder à pergunta que nos trouxe até aqui: Como fazer com que as máquinas enxerguem o mundo como nós?

## REFERÊNCIAS

BRADSKI, G. **The OpenCV Library**. Dr. Dobb's Journal of Software Tools, 2000.

CANNY, J. **A Computational Approach to Edge Detection**. In: IEEE Transactions on Pattern Analysis and Machine Intelligence, vol. PAMI-8, no. 6, pp. 679-698, Nov. 1986, doi: 10.1109/TPAMI.1986.4767851.

DENG, J. K. LI, M. DO, H. SU, L. Fei-Fei. **Construction and Analysis of a Large Scale Image Ontology**. In: Vision Sciences Society (VSS), 2009.

KINREDON. 2017 CS231n学习笔记(一)----计算机视觉历史简述 (Computer vision overview). **Blog CSDN**, 22 de outubro de 2018. Disponível em: <<https://blog.csdn.net/djh123456021/article/details/83272227>>. Acesso em: 20 maio 2025.

KRIZHEVSKY, Alex; SUTSKEVER, Ilya; HINTON, Geoffrey E. ImageNet classification with deep convolutional neural networks. **Communications of the ACM**. 60 (6): p. 84–90, 24 maio 2017.

KUMAR, Vaibhav. Hands-On Guide To Object Detection Using YOLO. **Analytics India Magazine**, 17 de junho de 2020. Disponível em: <<https://analyticsindiamag.com/deep-tech/hands-on-guide-to-object-detection-using-yolo/>>. Acesso em: 20 maio 2025.

LECUN, Yann; BENGIO, Yoshua. Convolutional networks for images, speech, and time series. **The handbook of brain theory and neural networks**. 3361.10 (1995): 1995.

LODUR, Felipe. O que é Processamento de Imagens? (e algumas aplicações). **Medium**, 19 de novembro de 2021. Disponível em: <<https://blog.codedojo.com.br/o-que-%C3%A9-processamento-de-imagens-e-algumas-aplica%C3%A7%C3%B5es-b7c71a6cfefb>>. Acesso em: 20 maio 2025.

LOWE, David G. Distinctive Image Features from Scale-Invariant Keypoints. **International Journal of Computer Vision**. 60 (2). p. 91–110, November 2004.

PIEMONTEZ. Identificação, Detecção, Reconhecimento e Segmentação de Imagem e Objetos. **Visão Computacional**, 2022. Disponível em: <<https://visaocomputacional.com.br/identificacao-deteccao-reconhecimento-e-segmentacao-de-imagem-e-objetos/>>. Acesso em: 20 maio 2025.

REDMON, Joseph. *et al.* **You only look once**: Unified, real-time object detection. Proceedings of the IEEE conference on computer vision and pattern recognition. 2016.

ROKKAM, Abhay. Decoding Digital Images: The Fundamentals of Grayscale and RGB. **Medium**, 18 de outubro de 2024. Disponível em: <<https://medium.com/@abhayrokkam/decoding-digital-images-the-fundamentals-of-grayscale-and-rgb-e16145d4df81>>. Acesso em: 20 maio 2025.

SIMONYAN, Karen; ZISSERMAN, Andrew. Very deep convolutional networks for large-scale image recognition. **arXiv** preprint arXiv:1409.1556 (2014).

SODAWHISKEY. Gatos preguiçosos estão dormindo no sofá confortável com um gesto engraçado. **Freepik**, 2025. Disponível em: <[https://br.freepik.com/fotos-premium/gatos-preguicosos-estao-dormindo-no-sofa-confortavel-com-um-gesto-engracado-conceito-de-animais-da-vida-domestica\\_31712276.htm](https://br.freepik.com/fotos-premium/gatos-preguicosos-estao-dormindo-no-sofa-confortavel-com-um-gesto-engracado-conceito-de-animais-da-vida-domestica_31712276.htm)>. Acesso em: 20 maio 2025.

SUTOYO, Rhio. Convolutional neural network for face recognition in mobile phones. **ResearchGate**, julho de 2019. Disponível em: <[https://www.researchgate.net/figure/CNN-architectures-and-its-error-rate-result-on-ILSVRC-1\\_fig1\\_358832743](https://www.researchgate.net/figure/CNN-architectures-and-its-error-rate-result-on-ILSVRC-1_fig1_358832743)>. Acesso em: 20 maio 2025.